

TECHNICAL DESIGN DOCUMENT

Project	Windows 95 Personal Portfolio Web App
Version	1.0 — Initial Draft
Date	May 21, 2026
Status	Draft — For Review

1. Project Overview

A single-page web application that fully emulates the Windows 95/98 desktop environment as a personal portfolio. All content lives inside draggable, resizable windows — identical in chrome to the original OS — rendered inside the browser. No backend is required for MVP; all portfolio data is static and embedded. The experience is entirely client-side.

The goal is to replace the typical portfolio page (hero → projects → contact) with an explorable environment. Visitors discover the owner's work, personality, and contact info at their own pace by interacting with a simulated operating system.

2. Tech Stack

Layer	Choice	Rationale
Framework	React 18 (Vite)	Component model maps perfectly to the window/app paradigm; fast HMR
Styling	CSS Modules + CSS Variables	Scoped styles; easy Win95 theme tokens; no runtime CSS-in-JS overhead
State Mgmt	Zustand	Lightweight; manages window registry, z-index stack, taskbar state globally
Canvas	HTML5 Canvas API	Native, zero-dependency drawing surface for MSPaint; full pixel control
Audio	Web Audio API + Howler.js	Howler abstracts cross-browser quirks; used for audio player & UI sounds
Icons	Custom pixel-art PNGs	Authentic Win95 icon aesthetic; 16x16 and 32x32 sprite sheets
Fonts	Perfect DOS VGA / px437	Period-accurate CRT terminal look; Google Fonts fallback: Courier Prime
Build Tool	Vite	Sub-second HMR; tree-shaking; minimal bundle for static deployment
Deployment	Netlify / GitHub Pages	Static hosting; no server required; free tier sufficient

3. Architecture

3.1 High-Level Component Tree

```
<App>
<BootScreen /> // BIOS POST → splash → fade in
<Desktop> // Teal wallpaper; icon surface
  <DesktopIcon × N /> // Double-click to open window
  <WindowManager> // Owns open window stack + z-index
    <Window × N> // Draggable / resizable shell
      <TitleBar />
      <MenuBar /> // Optional, per-app
      <AppContent /> // Routes to correct app component
      <StatusBar /> // Optional, per-app
    </Window>
  </WindowManager>
</Desktop>
<Taskbar>
  <StartButton />
  <StartMenu /> // Layered above taskbar on click
  <TaskbarButton × N /> // One per open window
  <SystemTray> // Live clock + volume icon
</SystemTray>
```

```
</Taskbar>
</App>
```

3.2 Window Manager — Core Subsystem

The WindowManager is the most critical piece of the application. It owns all open window state in a Zustand store and is responsible for focus, z-index ordering, drag/resize behavior, and taskbar synchronization.

Window State Shape

```
interface WindowState {
  id: string; // uuid — stable across re-renders
  appId: AppId; // Which app component to render
  title: string;
  isMinimized: boolean;
  isMaximized: boolean;
  zIndex: number; // Managed by topZIndex counter
  position: { x: number; y: number };
  size: { width: number; height: number };
  defaultSize: { width: number; height: number };
}
```

Z-Index Strategy

A monotonically incrementing counter **topZIndex** is stored in Zustand. Focusing any window sets its zIndex to **++topZIndex**. Base desktop z-index: 1. Windows: 100+. Taskbar: 9999. Start Menu: 10000. Modal dialogs (Shut Down, errors): 10500.

Drag Implementation

onMouseDown on the title bar records the cursor offset relative to the window's top-left corner. mousemove and mouseup are attached to the global window object (not the element) to handle fast cursor movement without losing the drag. Position is clamped so the title bar cannot leave the viewport. Uses Pointer Events API for stylus/touch support.

Resize Implementation

Eight invisible resize handles (4 corners + 4 edges) are absolutely positioned on the window border. Each handle encodes a resize direction. The same global pointer-capture pattern used for drag is applied. Minimum sizes are enforced per-app (e.g. MSPaint min 480×400, Contact min 360×320).

3.3 App Registry

A static manifest maps each AppId to metadata and the React component to render:

```
const APP_REGISTRY: Record<AppId, AppDef> = {
  aboutMe: { title: 'About Me', icon: 'user.png', component: AboutMe,
    defaultSize: { w: 540, h: 420 } },
  portfolio: { title: 'Portfolio', icon: 'folder.png', component: Portfolio,
    defaultSize: { w: 640, h: 500 } },
  contact: { title: 'Contact', icon: 'mail.png', component: Contact,
    defaultSize: { w: 400, h: 360 } },
  mspaint: { title: 'MS Paint', icon: 'paint.png', component: MSPaint,
    defaultSize: { w: 700, h: 520 } },
  audioplayer: { title: 'CD Player', icon: 'cdrom.png', component: AudioPlayer,
    defaultSize: { w: 320, h: 180 } },
  notepad: { title: 'Notepad', icon: 'notepad.png', component: Notepad,
    defaultSize: { w: 480, h: 380 } },
  minesweeper: { title: 'Minesweeper', icon: 'mine.png', component: Minesweeper,
    defaultSize: { w: 300, h: 360 } },
  solitaire: { title: 'Solitaire', icon: 'cards.png', component: Solitaire,
    defaultSize: { w: 600, h: 500 } },
}
```

```
ie: { title:'Internet Explorer', icon:'ie.png', component: FakeIE,  
  defaultSize: { w:620, h:460 } },  
recycle: { title:'Recycle Bin', icon:'trash.png', component: RecycleBin,  
  defaultSize: { w:360, h:280 } },  
};
```

3.4 MSPaint — Canvas Subsystem

MSPaint is the most complex individual app. It uses two stacked HTML5 canvases: a **display canvas** (visible) and an off-screen **buffer canvas** for undo snapshot management.

Paint State

```
interface PaintState {
  tool: 'pencil'|'brush'|'eraser'|'fill'|'line'|
  'rect'|'ellipse'|'text'|'eyedropper'|'select';
  primaryColor: string; // hex
  secondaryColor: string; // hex
  brushSize: number; // 1 | 3 | 5 | 8
  undoStack: ImageData[]; // max 20 frames
  redoStack: ImageData[];
}
```

Flood Fill Algorithm

An iterative BFS (breadth-first search) flood fill — NOT recursive — is used to prevent call-stack overflow on large canvas areas. The algorithm samples the full ImageData array, maintains a typed Uint8Array visited mask for O(1) lookup, and matches pixels within a color tolerance of 30 (Euclidean distance in RGB space). A queue-based approach processes ~600k pixels in under 16ms on modern hardware.

Undo/Redo

On every mousedown (stroke begin), the current ImageData is pushed to undoStack. Ctrl+Z pops undoStack, pushes current state to redoStack, and restores the snapshot. Stack is capped at 20 frames. ImageData snapshots (~900KB each for 600x400) are stored in memory only — no disk I/O.

3.5 Audio Player — CD Player Subsystem

Styled as a Windows 98 CD Player widget. Backed by the Web Audio API via Howler.js for cross-browser compatibility.

```
interface AudioPlayerState {
  playlist: Track[]; // { title, artist, src }[]
  currentIdx: number;
  isPlaying: boolean;
  volume: number; // 0.0 - 1.0
  currentTime: number; // seconds
  duration: number;
  isShuffle: boolean;
  isRepeat: 'none'|'one'|'all';
}
```

Audio files are hosted as static assets (MP3/OGG). The portfolio owner provides royalty-free tracks or their own music. Browser autoplay policy requires the first play to be triggered by a direct user gesture — the Play button satisfies this.

3.6 Taskbar & Start Menu

- **Taskbar:** Fixed to bottom of viewport. Height: 28px. Contains Start button (left), window buttons (scrollable center), system tray (right). Window buttons truncate title at ~120px with ellipsis. Active window button appears inset (sunken border).
- **Start Menu:** Positioned div anchored above the Start button. Programs submenu renders as an adjacent panel. Items: Programs (all apps), Settings (dummy), Help (opens Notepad with easter-egg readme), Shut Down.
- **Shut Down Dialog:** A modal (not a Window) with radio options — Shut Down / Restart / Suspend. Shut Down triggers a CSS fade-to-black, then 'It is now safe to turn off your computer.' Any click after 2s reloads the page.
- **System Tray Clock:** A single setInterval(1000) in a top-level effect. Zustand stores the Date. Only the Clock component subscribes and re-renders — no unnecessary repaints.

4. Win95 Visual System

4.1 CSS Design Tokens

```
:root {
--win95-desktop: #008080; /* Classic teal */
--win95-window-bg: #c0c0c0; /* Silver */
--win95-titlebar-active: #000080; /* Dark blue */
--win95-titlebar-inactive:#808080;
--win95-titlebar-text: #ffffff;
--win95-button-face: #c0c0c0;
--win95-button-shadow: #808080;
--win95-button-dark: #000000;
--win95-button-highlight: #ffffff;
--win95-button-light: #d0d0d0;

/* 3D bevel border system */
--border-raised: inset -1px -1px #000, inset 1px 1px #fff,
inset -2px -2px #808080, inset 2px 2px #d0d0d0;
--border-sunken: inset -1px -1px #fff, inset 1px 1px #000,
inset -2px -2px #d0d0d0, inset 2px 2px #808080;
--border-field: inset -1px -1px #d0d0d0, inset 1px 1px #808080,
inset -2px -2px #c0c0c0, inset 2px 2px #000;

--font-system: 'px437', 'Courier Prime', monospace;
--font-size-base: 11px;
}
```

4.2 Window Chrome Anatomy

Region	Description
Title Bar	Active: --win95-titlebar-active gradient. Contains 16x16 app icon, title text (truncated), and 3 control buttons (⏏, ■, ✕) styled with --win95-button-face and --win95-button-shadow.
Menu Bar	Optional. Flat bar; hover reveals dropdown panels. Items are plain text buttons. Keyboard navigation via Alt + letter.
Content Area	Background: --win95-window-bg. Inset border (--border-field) on inner content panes (listboxes, textareas, canvas areas).
Status Bar	Optional 1-row footer. 2px top border. Left-aligned status text. Right-aligned secondary info. Matches MSPaint, Explorer.
Resize Handles	8 invisible hit-areas (4px wide) on all edges and corners. cursor CSS changes per direction.

5. State Management (Zustand)

Store	Key Actions
windowStore	openWindow(appId) · closeWindow(id) · focusWindow(id) · minimizeWindow(id) · maximizeWindow(id) · moveWindow(id, x, y)
desktopStore	selectedIcon: AppId null · setSelectedIcon(id) · clearSelection()
systemStore	volume: 0-100 · time: Date (ticks every 1s) · bootComplete: boolean · shutdownState: 'running' 'shutdown' 'restarting'
audioStore	playlist · currentIdx · isPlaying · volume · currentTime · duration · shuffle · repeat

6. Boot Sequence

Step 1: Black Screen

Immediate on load. No content visible.

Step 2: BIOS POST Text

White monospace text scrolls on black: RAM check, peripheral detection, BIOS version — all fake but believable. ~2.5s via cascading `setTimeout`.

Step 3: 'Starting Windows 95...'

Full-screen message + the classic segmented progress bar animation (colored blocks cycling). ~1.5s.

Step 4: Splash Screen

Windows 95 logo (CSS-drawn or SVG) + indeterminate progress bar. ~2s.

Step 5: Desktop Fade-In

opacity 0 → 1 over 600ms. Startup sound plays here (Howler, user-gesture gated).

Implementation: A `<BootScreen>` component with sequential timed state transitions. Once `bootComplete = true` in `systemStore`, `BootScreen` unmounts and `Desktop` mounts. A skip mechanism (click/keypress) advances immediately for repeat visitors.

7. Performance Considerations

- **MSPaint Canvas:** Event-driven drawing only — no `requestAnimationFrame` loop. Dramatically reduces CPU usage for idle windows.
 - **Minimized Windows:** All windows stay mounted when minimized (`display:none` via CSS). Preserves app state (especially canvas pixel data) and avoids remount cost.
 - **Undo Snapshots:** `ImageData` pushed only on mouseup (stroke commit), not every mousemove event. Capped at 20 frames.
 - **Clock:** Single `setInterval(1000)` at the App root. Only the Clock component subscribes to the time slice — no full re-renders.
 - **Audio:** Howler.js lazy-loads audio files on first play, not on app mount. Prevents blocking the boot sequence.
 - **Code Splitting:** Each app component is a `React.lazy()` import. Heavy apps (Solitaire, Minesweeper) are only bundled when first opened.
-

8. Responsive & Mobile

The Win95 desktop experience is inherently desktop-first. The minimum comfortable viewport is 1024×600. Behavior below this:

- **< 768px wide:** A Win95-styled dialog overlay reads 'This site is best viewed on a 1024×768 or larger display.' Dismissible. Desktop still accessible behind it.
 - **Touch support:** `touchstart/move/end` are mapped to synthetic `MouseEvent`s in the drag and MSPaint pointer handlers. Pinch-to-zoom is suppressed (meta viewport no-user-scalable).
 - **Small viewports:** Desktop div has a fixed minimum width of 1024px with `overflow:auto` on the outer container so the experience scrolls rather than breaks.
-

9. File Structure

```
src/  
components/  
Desktop/ Desktop.tsx DesktopIcon.tsx  
Window/ Window.tsx TitleBar.tsx WindowControls.tsx ResizeHandle.tsx  
Taskbar/ Taskbar.tsx StartButton.tsx StartMenu.tsx SystemTray.tsx  
BootScreen/ BootScreen.tsx  
ui/ Button.tsx Input.tsx Dialog.tsx ProgressBar.tsx  
apps/  
AboutMe/ AboutMe.tsx  
Portfolio/ Portfolio.tsx ProjectDetail.tsx  
Contact/ Contact.tsx  
MSPaint/ MSPaint.tsx Canvas.tsx Toolbar.tsx  
ColorPalette.tsx floodFill.ts  
AudioPlayer/ AudioPlayer.tsx Playlist.tsx visualizer.ts  
Notepad/ Notepad.tsx  
Minesweeper/ Minesweeper.tsx logic.ts  
Solitaire/ Solitaire.tsx cardEngine.ts  
FakeIE/ FakeIE.tsx pages/  
RecycleBin/ RecycleBin.tsx  
store/  
windowStore.ts desktopStore.ts systemStore.ts audioStore.ts  
registry/  
appRegistry.ts  
styles/  
tokens.css global.css win95.css  
assets/  
icons/ sounds/ wallpaper.bmp music/  
main.tsx
```

10. Development Milestones

Phase	Deliverable	Effort
1	Vite + Zustand scaffold; CSS token system; Window shell (drag / resize / minimize / maximize / close)	2 days
2	Taskbar, Start Menu, Desktop Icons, Boot Sequence	2 days
3	About Me, Contact, Portfolio apps (static content, full layout)	1.5 days
4	MSPaint — canvas, all tools, palette, undo/redo	3 days
5	Audio Player — CD Player widget, Howler.js integration, playlist	1.5 days
6	Notepad, Fake IE, Recycle Bin	1 day
7	Minesweeper (full game logic)	1.5 days
8	Solitaire (full Klondike + bounce animation)	2 days
9	Polish: sounds, boot animation, easter eggs, mobile warning	2 days
10	QA, accessibility, deployment pipeline	1 day
Total		~18 days

Out of Scope (MVP)

- Saving MSPaint files to disk
- Actual file system simulation
- Real external URL browsing in Fake IE
- Multi-user state or any backend persistence
- Drag-to-reposition desktop icons
- True window snapping / Aero-style behavior